

Lab

Static fields and methods

Lab: Game — static fields and methods

Objective

In this lab, you will:

- Further your understanding of object-oriented programming.
- Create a graphical application instead of a Console app.

Part 1

You'll be creating a graphical application, which is an exciting new challenge. While this will involve copying and pasting several lines of code, you don't need to fully understand how the graphics code functions.

However, it's worth noting that the code isn't overly complex, so if you're curious, you can take the time to explore and investigate it on your own.

1. Back in the **labs** project, create a new class called **Game** in a package named **oo2**.
2. Create an instance of Game in the **main()** method. This will run the Game's constructor code which you will create later.
3. To begin, you'll need a canvas to draw on and display your animation. In this example, you'll create several ball objects bouncing around within a rectangle (referred to as 'the world').

Feel free to get creative with this project later, but for now, it's important to follow the provided steps. The primary goal is to explore and understand object-oriented programming, not to focus on animation or graphic design.

Start by deleting any existing code in the Game class. Then, copy and paste the code below. You'll notice that there are a few comments included in the code, which act as placeholders for sections you'll write later.

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.Timer;
import javax.swing.*;

public class Game extends Canvas {
    public static void main(String[] args) {
        new Game();
    }

    // create an array of 3 balls
    // call this array balls
```

```
Game() {
    JFrame frame = new JFrame();
    this.setSize(400, 400);
    frame.add(this);
    frame.pack();
    frame.setVisible(true);

    Timer t = new Timer();
    TimerTask tt = new TimerTask() {
        @Override
        public void run() {
            draw();
        }
    };

    t.schedule(tt, 0, 50);
```

Creates a form

Creates a timer to call
the draw() method every
50ms (1/20 sec)

```
frame.addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        t.cancel();
        tt.cancel();
    }
});

public void draw() {
    this.repaint();
}
```

Runs when
the window
is closed

Invokes a repaint
which will run the
paint() method

```

public void paint(Graphics g) {
    // call the move() method of each balls
    // Tip: use an enhanced for loop to pick
    //       each ball in the balls array.

    // draw each ball in balls array
    // Tip: See the example below
    g.setColor(Color.RED);
    g.fillRect(0, 0, 300, 300);
    g.setColor(Color.blue);
    g.fillOval(x, y, width, height);
}
}

```

4. Add **requires java.desktop;** to your **module-info.java** file:

```

module <your module name> {
    requires java.desktop;
}

```

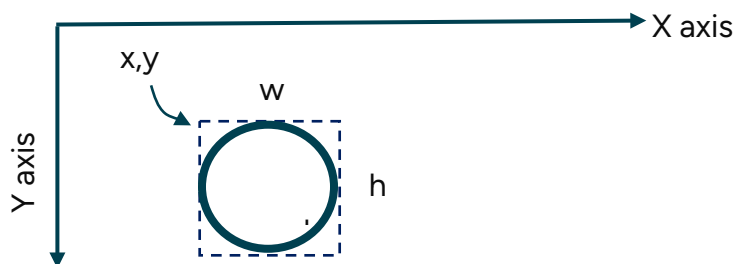
5. Run the application to see if your code works as expected.

6. Create a class called **Ball** with the following fields.

```

public int x, y, w, h;
private int dirX, dirY;

```



A ball's dimension is represented as a rectangle. X and y are the top coordinate and w and h are the width and height of the ball's rectangle.

dirX and dirY are the amount by which each ball moves in the x and y direction.

The coordinate system is perhaps unlike what you're familiar with as it is upside-down!

X increases to the right and Y increases towards the bottom.

7. Create a constructor to set every field (x,y,w,h,dirX,dirY).

8. To practice using constructor chaining, create another constructor to set just x,y,w,h but not the dirX and dirY.

Use constructor chaining to call the first constructor with dirX=1 and dirY=1.

9. Create a method called **move()** in the **Ball** class.

In this method, increase x by dirX and y by dirY and then follow a series of tests to make sure each ball stay within the world which is set in the **paint** event as 300px wide, 300px high, and starts at x=0,y=0.

If (x < 0) then set x=0 and change the sign of dirX.

Tip: **dirX = -dirX**

Do the same for the y coordinate (when it goes below zero) but this time changing dirY.

If (x > 300-width of the ball) then set x=300-width of the ball and reverse the sign of dirX.

Do the same for the y coordinate (taking the height of the ball into account).

10. Back in the Game class, view the comments and perform those tasks (create three balls, call move()... and draw each ball).

Tip: The paint method can draw a circle using this code:

```
g.drawOval( b.x, b.y, b.w, b.h);  
// where b is a ball reference
```

11. If everything works, you should see three objects bouncing on your screen. If you have a need for speed then increase the dirX and dirY values!

Part 2 — Using the **static** keyword

In this section, you will explore the use of the static keyword. Defining the world dimensions as fixed values, such as 300 pixels for both height and width, is not an ideal approach.

Since all the balls share the same world coordinates, this scenario presents an excellent opportunity to utilise the static keyword. By doing so, you ensure that the world dimensions are shared across all instances, reflecting their universal nature within the program.

1. Create a static int field in Ball called **worldW** and another called **worldH** to represent the width and height of the world. You should see three balls bouncing!

2. Create static method as

```
public static void setWorld(int w, int h)
```

to set the static *worldW* and *worldH* values.

3. In the `Game()` constructor, just before the line `Timer t = new Timer();` invoke the `setWorld()` method of the `Ball` class to set the world's width and height for every `Ball`.

4. Change the `move()` method in the `Ball` class to use the `Ball.worldH` and `Ball.worldW` values instead of 300px. You'll also need to change the `paint()` method to use the new static values (to draw the world's rectangle).

5. Run and test your code.

Optional challenge lab (if there is time)

More on statics

Lab: Exploring static

Objective

In this lab, you will implement the factory pattern. You will get a 'Registration Plate' from a 'Registration Plate Factory' class.

Step-by-step

In this section of the lab, you will beef up a `Vehicle` class to add both a static member and an instance member and use static members of a `Factory` class.

You'll create a few vehicles and then race them to see which one wins!

6. Add a new class to the labs project called **Program** with **main()** method in a package called **lab6**.
7. Create a class called **Vehicle** in the **lab3** package, with integer fields called **speed** and **lane**.
Also record the distance travelled using an int variable called **distanceTravelled**.
8. Create a constructor to set the speed and lane fields.
9. Add two methods to the `Vehicle` class called **accelerate** and **brake**.

```
void accelerate(int amount)
```

The accelerate method will increase the speed but never more than 200!

It also adds to the **distanceTravelled** the int **amount**.

```
void brake(int amount)
```

This will reduce the speed by amount.

10. Add another method as **String** `getDetails()` to get the vehicle's details which are **speed**, **lane**, **distanceTravelled**, and **plate** (see below).
11. Every vehicle has a registration **plate** which is a complex object (has information about city, country, and the year of registration) which should be defined as a class.
 - a) Create a separate class called **RegistrationPlate**.

- b) Give this class a private field called **number**. This is the registration plate number which in real-life is a made of a complex code.
Provide a getter method to read the value of this field.
- c) Feel free to add getters for city, country, and the year of registration which can be deduced from the plate number, but this is not required for this exercise.
- d) Create a constructor to set the **registration Plate number field**. This method has no parameters because you will set the **registration Plate number field by using a factory** (seen later).
- e) Create a getter method for *the* **registration Plate number field**.

12. Add a new field to the **Vehicle** class called **registrationPlate** of type `RegistrationPlate`.

Like:

RegistrationPlate registrationPlate;

You'll set this field using a factory pattern.

Please **do not** instantiate it here.

13. Let's create a factory class which creates instances of `RegistrationPlate`.

14. Create a separate class called **RegistrationPlateFactory**.

15. Add the following array of reg numbers to `RegistrationPlateFactory`.

```
private static String[] registrationPlates = {  
    "LA05 XYZ", // London  
    "NG58 JKL", // Nottingham  
    "MA13 HYP", // Manchester  
    "BX06 TUV", // Birmingham  
    "CF67 QRS", // Cardiff  
    "YS09 PQR", // Yorkshire  
    "WD21 NML", // Exeter area (Western region)  
    "KR16 FGH", // Northampton  
    "SJ04 UVW", // Glasgow  
    "LV22 ABC", // Liverpool  
    "WX19 XYZ" // Bristol  
};
```

16. Note, The array is **static** because it will be accessed by a static method.

17. Create a **static** method called **getNextRegistrationPlate()** This method should return an instance of **RegistrationPlate**. To create an instance, you'll need the next registration plate from the **registrationPlates** array.

How would you get the next regPlate index?

18. Back in the Vehicle class's constructor, assign a registration plate to the Vehicle using the **RegistrationPlateFactory** class.

Tip:

```
registrationPlate = RegistrationPlateFactory. getNextRegistrationPlate();
```

19. In the main() method, Create an ArrayList of Vehicles, populated with three new Vehicles.

20. Print details of the vehicles created in the above step.

Please make sure the plates are correctly assigned.

Enhancing the Vehicle class

21. How would you count the number of vehicles created?

I know you created three, but what if different parts of your program created Vehicles? Create a static method called getCount() to return the count of vehicles.

Tip: Vehicle's constructor is invoked whenever a vehicle is created. Please see the slides for more help.

Writing code to race the Vehicles!

22. Return to the main method.

- a) The initial speed of the Vehicles is zero.
- b) Make sure they are placed in lanes 1, 2, and 3.

23. Write a while loop to race the cars (accelerate them) until the distance travelled by one of them is more than 1000.

You'll accelerate each vehicle by a random number between 1-10 by using the following code:

```
Random rand = new Random();
```

```
int n = rand.nextInt(10)+1;
```

24. Display details of each vehicle as they accelerate on each iteration of the while loop.

25. As soon as a vehicle has travelled 1000m or more, announce it as winner and break out of the loop.
26. You can get creative and assign drivers to each vehicle.



QA.com